TITLE OF THE PROJECT
LEARNING MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

Ashwin Krishna.V

[727824TUCS031]

In Partial fulfilment for the award of the degree

of

Bachelor of Engineering

IN

COMPUTER SCIENCE AND ENGINEERING

SRI KRISHNA COLLEGE OF ENGINEERIN TECHNOLOGY

An Autonomous Institution | Approved by AICTE | Affiliated by Anna University | Accredited by NAAC with A++ Grade, Kuniyamuthur, Coimbatore – 641008.

Month & Year

JULY 2026

SRI KRISHNA COLLEGE OF ENGINEERING TECHNOLOGY
An Autonomous Institution | Approved by AICTE | Affiliated to Anna University |
Acccredited by NAAC with A++ Grade, Kuniyamuthur, Coimbatore – 641008.

BONAFIDE CERTIFICATE

Certified that this project **"LEARNING MANAGEMENT SYSTEM"** is the Bonafide work of **"Ashwin Krishna V"** who carried out the project work under my supervision.

SIGNATURE **SIGNATURE**

HEAD OF THE DEPARTMENT SUPERVISOR

Submitted for the Project Viva-voce examination held on _____.

INTERNAL EXAMINER EXTERNAL EXAMINER

ABSTRACT

The **Learning Management System** (LMS) Lite is a web-based educational platform developed to streamline the delivery, management, and assessment of online learning for institutions, instructors, and students. The primary objective of the project is to provide a secure, role-based platform where administrators, instructors, content creators, teaching assistants, and students can manage courses, learning content, assessments, and academic progress from a single system, eliminating the fragmented and manual processes traditionally used for course and assessment management.

The **backend** of the application is developed using the Spring Boot framework and follows a layered architecture consisting of the Controller, Service, Repository, and Model layers. Spring Security with JSON Web Token (JWT) based authentication is used to secure all protected operations, while a comprehensive Role-Based Access Control (RBAC) system governs the permissions available to each user type. RESTful APIs are implemented to perform CRUD (Create, Read, Update, Delete) operations for courses, enrollments, learning content, assessments, grades, discussions, and notifications. Spring Data JPA is used for database interaction, allowing the system to work with any JPA-compatible relational database for the secure storage of academic and user data.

The React.js frontend consumes these secured REST APIs to present role-specific dashboards for course management, content delivery, assignment submission, grading, and learning analytics. The system enforces data validation, FERPA-aligned handling of student records, and real-time progress tracking, while supporting integration points for institutional authentication systems, LTI-compliant tools, SCORM content packages, video conferencing, and plagiarism detection services. Automated and unit testing using JUnit, along with React Testing Library on the frontend, is used to verify the correctness of the implemented modules.

Overall, the LMS Lite system provides a reliable, secure, and scalable foundation for managing the complete academic lifecycle of a course — from creation and enrollment through content delivery, assessment, grading, and analytics — while establishing a strong base for future enhancements such as predictive learning analytics, AI-powered content recommendations, mobile applications, and deeper integration with institutional student information systems.

TABLE OF CONTENTS

S. No.	Table of Contents
1	Abstract
2	Table of Contents
3	List of Figures
4	List of Symbols, Abbreviations and Nomenclature
5	Chapter 1 - Introduction
6	Chapter 2 - Problem Statement
7	Chapter 3 - Objectives
8	Chapter 4 - Existing System
9	Chapter 5 - Proposed System
10	Chapter 6 - System Architecture
11	Chapter 7 - Technology Stack
12	Chapter 8 - UML Diagrams
13	Chapter 9 - Backend Implementation
14	Chapter 10 - API Endpoints
15	Chapter 11 - Testing
16	Chapter 12 - Software Requirements
17	Chapter 13 - Sample JSON Request & Response
18	Chapter 14 - Conclusion
19	Chapter 15 - Future Enhancements

LIST OF FIGURES

Charts & Graphs

- Comparison of Traditional and LMS-based Course Delivery (Bar chart illustrating improvements over manual processes)
- Challenges in Traditional Course & Assessment Management (Bar chart showing limitations of manual handling)
- Major Functionalities of the Proposed System (Donut chart showing core feature distribution)
- Technology Stack Distribution (Donut chart showing core technologies used)
- User Role Distribution and Permission Levels (Donut chart showing role-based access scope)

Architecture & UML Diagrams

- Project Directory Structure (Source code folder hierarchy src/main/... under the System Architecture section)
- Use Case Diagram (Showing interactions between Student, Instructor, Content Creator, Teaching Assistant, Administrator and the system)
- ER Diagram (Showing relationships between Users, Courses, Enrollments, CourseContent, Assessments, Grades, and Notifications)
- Class Diagram (Showing Controller, Service, Repository, and Model classes for the Course and Assessment modules)

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
JPA	Java Persistence API
JWT	JSON Web Token
RBAC	Role-Based Access Control
REST	Representational State Transfer
LMS	Learning Management System
SCORM	Sharable Content Object Reference Model
LTI	Learning Tools Interoperability
FERPA	Family Educational Rights and Privacy Act
GPA	Grade Point Average
ORM	Object Relational Mapping
HTTP	Hyper Text Transfer Protocol
JSON	JavaScript Object Notation
SQL	Structured Query Language
WCAG	Web Content Accessibility Guidelines
MOOC	Massive Open Online Course

CHAPTER - 1

INTRODUCTION

Educational institutions and training organisations increasingly rely on digital platforms to deliver course content, manage assessments, and track student progress. Traditional approaches that combine spreadsheets, email, shared drives, and standalone quiz tools are difficult to secure, hard to scale, and provide little visibility into how students are actually progressing through a course. A unified Learning Management System addresses this by bringing course delivery, enrollment, assessment, grading, communication, and analytics into a single, role-aware platform.

The Learning Management System (LMS) Lite is a web-based application developed to provide educational institutions with a streamlined, secure, and efficient platform for delivering online learning experiences. The system enables instructors to create and manage courses, content creators to build multimedia learning material, teaching assistants to support grading and student queries, students to enroll in courses and submit assessments, and administrators to oversee the entire platform — all while maintaining strict role-based access control and educational data protection standards.

The project follows a client-server architecture in which the frontend is developed using React.js and the backend is implemented using Spring Boot with Spring Security. The frontend is responsible for presenting role-specific dashboards, course catalogues, content viewers, and assessment interfaces, while the backend processes requests, enforces authorization rules, performs business logic, manages database operations, and exposes secured REST APIs. Communication between the frontend and backend takes place over HTTPS using JSON-formatted REST APIs, with JWT tokens used to authenticate every protected request.

The backend follows a layered architecture consisting of the Controller, Service, Repository, and Model layers, which separates request handling, business logic, and persistence concerns. Spring Data JPA is used for object-relational mapping and efficient interaction with the underlying relational database, and the schema is designed to be database-agnostic so that the system can be deployed against any JPA-compatible database engine.

The system implements a five-tier role hierarchy — System Administrator, Instructor, Content Creator, Teaching Assistant, and Student — in addition to limited Guest access, so that every operation in the platform, from creating a course to modifying a grade, is governed by an explicit permission model. Beyond core CRUD operations, the platform supports assessment creation and automated grading, learning progress analytics, communication and collaboration tools, a notification system, and institutional reporting, giving the platform the depth required for real academic use rather than a simple content repository.

Overall, the LMS Lite platform provides a reliable, secure, and extensible foundation for managing the complete academic lifecycle of a course, reducing the manual effort involved in course administration while giving instructors and administrators the visibility they need to support student success.

CHAPTER - 2

PROBLEM STATEMENT

Instructors and institutions that rely on disconnected tools — spreadsheets for grade tracking, email for announcements, shared drives for content, and separate quiz applications for assessments — face recurring problems: content and grades are scattered across systems, there is no single source of truth for a student's progress, access control is inconsistent or absent, and academic data is not handled in a manner consistent with student-privacy expectations such as FERPA.

The LMS Lite project addresses this by providing a single web-based platform that unifies course creation, content delivery, enrollment, assessment, grading, communication, and analytics behind a secure, role-based access model. The system follows a client-server architecture in which the React.js frontend communicates with a Spring Boot backend through RESTful, JWT-secured APIs. The backend is responsible for enforcing role-based permissions, validating academic data, persisting course and assessment records, and generating the analytics instructors and administrators need to monitor learning outcomes.

The project aims to reduce the administrative overhead of running a course, ensure that only authorized users can view or modify sensitive academic records, and give students a consistent, trackable view of their enrolled courses, assignments, and grades — while providing a technical foundation (REST APIs, RBAC, database schema) that can be extended with additional institutional integrations in the future.

CHAPTER - 3

OBJECTIVES

The key objectives of the Learning Management System (LMS) Lite project are:

- To design and implement a secure user registration and JWT-based authentication system supporting multiple educational roles.
- To build a role-based access control (RBAC) system that restricts course, content, and grade operations to appropriately permitted users.
- To provide instructors and content creators with tools to create, structure, and publish course content, including multimedia material.
- To implement a comprehensive assessment engine supporting quiz creation, timed assessments, automated grading, and rubric-based evaluation.
- To track student progress through course completion, assignment submission, and learning analytics dashboards.
- To provide instructor-student communication tools, including discussion forums, messaging, and announcements.
- To implement institutional reporting and analytics for enrollment trends, course effectiveness, and academic performance.
- To ensure the platform meets performance, reliability, security, and FERPA compliance requirements suitable for academic use.
- To design a modular, layered backend architecture that supports future integration with SIS, LTI, and SCORM-based tools.

CHAPTER - 4

EXISTING SYSTEM

Many small institutions and independent instructors currently manage courses using a combination of general-purpose tools: course material is shared through email attachments or cloud drive links, assignments are collected manually, grades are recorded in spreadsheets, and announcements are sent through group messaging apps or email. There is no centralized system that ties a student's enrollment, submissions, and grades together, and access to sensitive records is rarely restricted by a defined permission model.

Limitations of the Existing System

- No centralized platform to manage course content, enrollment, and grading together.
- Lack of role-based access control, so sensitive academic data is not consistently protected.
- Manual, error-prone grading and grade-record keeping with no automated grade calculation.
- No structured way to track student progress, engagement, or at-risk learners.
- Assessment tools (quizzes, timed tests, anti-cheating measures) are either absent or handled by unrelated third-party tools.
- Communication is fragmented across email, chat apps, and verbal announcements.
- No audit trail or compliance mechanism for handling student educational records.
- Limited or no reporting on course effectiveness, completion rates, or institutional performance.

CHAPTER - 5

PROPOSED SYSTEM

The proposed Learning Management System (LMS) Lite is a web-based application developed using React.js and Spring Boot that unifies course management, content delivery, assessment, grading, and learning analytics behind a single, role-based platform. Administrators, instructors, content creators, teaching assistants, and students each interact with a dashboard scoped to their permissions, while a JWT-secured REST API layer enforces access control on every request.

Features of the Proposed System

- Secure JWT-based authentication with role-specific session durations and refresh tokens.
- Role-Based Access Control (RBAC) governing course, content, grade, and administrative operations.
- Course creation and structured content management with modules, lessons, and multimedia support.
- Enrollment and learning-path management with prerequisite and capacity handling.
- Assessment engine supporting quizzes, assignments, rubric-based grading, and automated scoring.
- Real-time progress tracking and learning analytics dashboards for students and instructors.
- Discussion forums, messaging, and announcement tools for instructor-student communication.
- Institutional reporting on enrollment, course effectiveness, and academic performance.
- FERPA-aligned handling of student records with encryption, audit logging, and access controls.
- RESTful APIs designed for integration with LTI tools, SCORM content, and institutional SIS platforms.

CHAPTER - 6

SYSTEM ARCHITECTURE

```
lmslite-springapp
│
├── src
│   └── main
│       ├── java
│       │   └── com.lmslite.springapp
│       │       ├── config
│       │       │   ├── SecurityConfig.java
│       │       │   └── CorsConfig.java
│       │       ├── controller
│       │       │   ├── AuthController.java
│       │       │   ├── CourseController.java
│       │       │   ├── EnrollmentController.java
│       │       │   ├── AssessmentController.java
│       │       │   └── UserController.java
│       │       ├── model
│       │       │   ├── User.java
│       │       │   ├── Course.java
│       │       │   ├── Enrollment.java
│       │       │   ├── CourseContent.java
│       │       │   ├── Assessment.java
│       │       │   ├── AssessmentSubmission.java
│       │       │   ├── Grade.java
│       │       │   └── Notification.java
│       │       ├── repository
│       │       │   └── ... (JpaRepository interfaces per entity)
│       │       ├── service
│       │       │   └── ... (business logic per module)
│       │       ├── security
│       │       │   ├── JwtUtil.java
│       │       │   └── JwtAuthFilter.java
│       │       └── SpringappApplication.java
│       └── resources
```

```
|       └── application.properties
└── pom.xml
```

Explanation

The LMS Lite platform follows a three-tier client-server architecture consisting of the React.js frontend, the Spring Boot backend, and a JPA-compatible relational database. This separation keeps presentation, business logic, and persistence concerns independent, which improves modularity, scalability, and maintainability.

The React.js frontend renders role-based dashboards for administrators, instructors, content creators, teaching assistants, and students. It handles course browsing, content viewing, assessment submission, and progress visualization, and communicates with the backend exclusively through RESTful APIs secured with JWT bearer tokens.

The Spring Boot backend is the core processing layer. Incoming requests first pass through a JWT authentication filter and Spring Security's authorization rules, which enforce the role-based access control matrix before a request reaches the Controller layer. The Controller layer delegates to the Service layer, which implements business rules such as enrollment capacity checks, grade calculation, and assessment scoring, before calling the Repository layer to persist or retrieve data through Spring Data JPA.

The database layer stores all academic entities — users, courses, enrollments, course content, assessments, submissions, grades, discussions, and notifications — and is designed to be database-agnostic through JPA, allowing deployment against different relational database engines without changing application code.

CHAPTER - 7

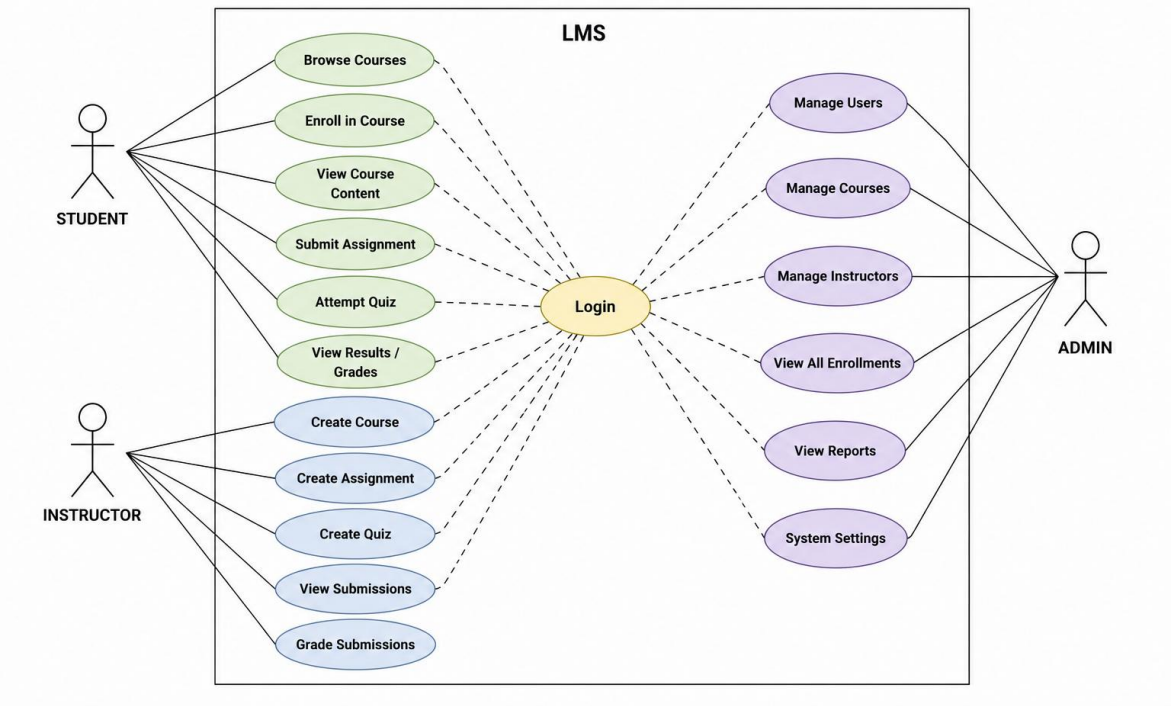
TECHNOLOGY STACK

Tool / Technology	Purpose
Spring Boot 3.x	Backend application framework
Spring Security	Authentication and authorization
JWT (JJWT)	Stateless token-based authentication
Spring Data JPA	Database operations and ORM
JPA-compatible RDBMS	Persistent storage of academic data
React.js	Frontend user interface
Maven	Dependency management and build tool
JUnit	Backend unit and integration testing
React Testing Library	Frontend component testing
Docker	Containerization for deployment
Git	Version control

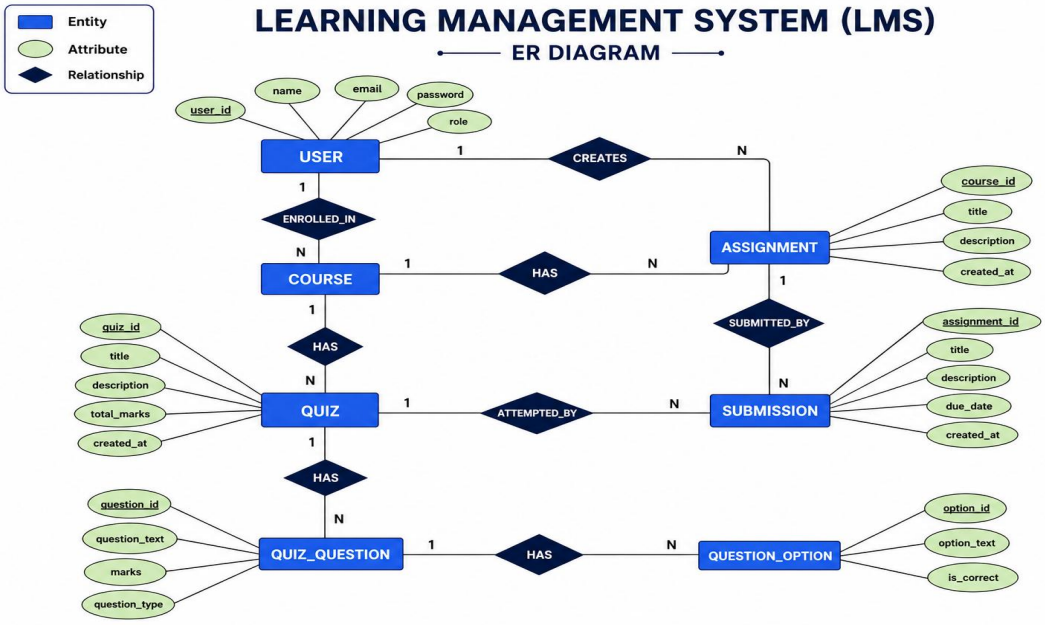
CHAPTER - 8

UML DIAGRAMS

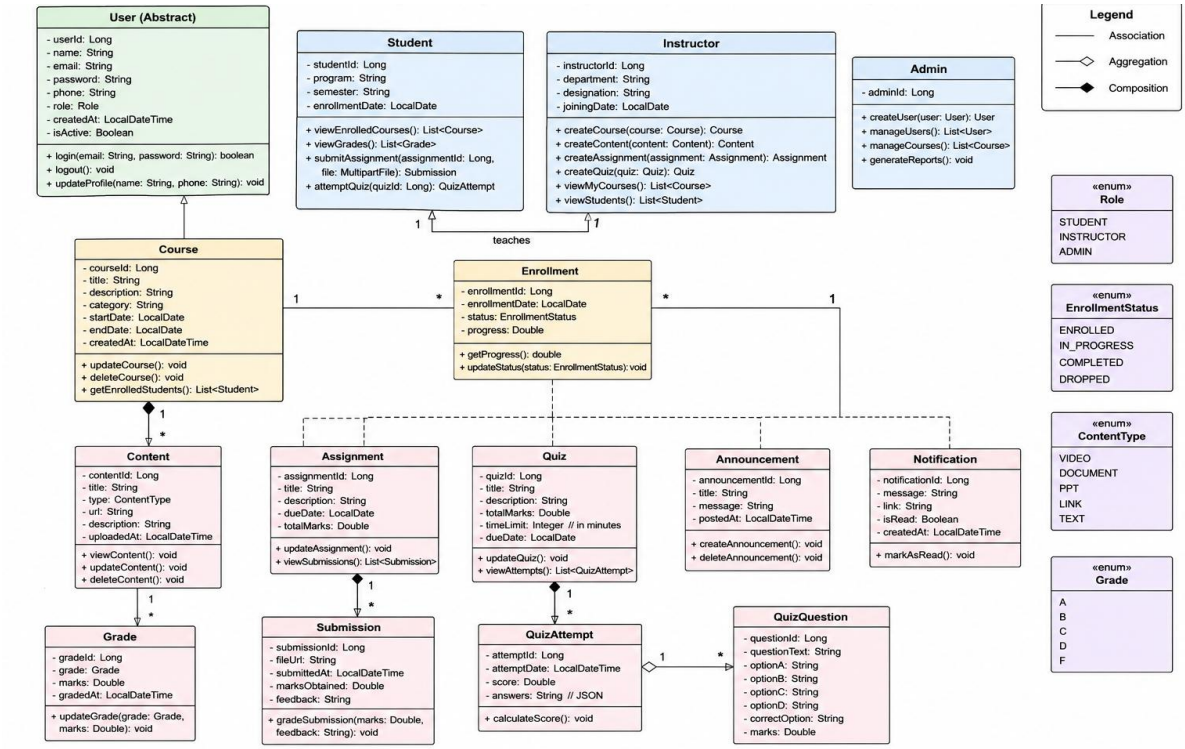
Use Case Diagram



ER Diagram



Class Diagram



CHAPTER - 9

BACKEND IMPLEMENTATION

The backend of the LMS Lite platform was developed using Spring Boot, Spring Security, Spring Data JPA, and JWT-based authentication. It follows a layered architecture consisting of the Model, Repository, Service, and Controller layers, with a dedicated security layer handling authentication and authorization.

User & Role Model

Stores the identity and role of every platform user.

- Full name, username, email, and hashed password
- Role (Student, Instructor, Content Creator, Teaching Assistant, Administrator)
- Student ID / employee ID, institution ID, and department
- Account status, last login timestamp, and creation date

Course & Content Model

Stores course metadata and the structured learning content within it.

- Course code, title, description, credits, schedule, and enrollment capacity
- Course status (Draft, Published, Archived, Cancelled)
- Hierarchical content items (modules, lessons, multimedia) linked to a course

Enrollment, Assessment & Grade Model

- Enrollment records linking a student to a course, with status and completion date
- Assessment definitions (quizzes, assignments) with type, points, due date, and time limit
- Assessment submissions with score, attempt count, and instructor feedback
- Grade records aggregating scores per student, course, and assessment

Repository Layer

Implemented using Spring Data JPA's JpaRepository interfaces, one per entity, responsible for:

- Persisting and retrieving Users, Courses, Enrollments, CourseContent, and Assessments
- Custom finder methods such as findByInstructorId, findByStudentId, and findByCourseId
- Delegating all SQL generation and ORM mapping to Spring Data JPA

Service Layer

Contains the business logic of the platform, including:

- Enrollment capacity and prerequisite validation before enrolling a student
- Automated grading for objective assessment questions and grade-book aggregation
- Role-based filtering so users can only retrieve data permitted by their role
- Progress and analytics calculations for student dashboards and instructor reports

Controller Layer

Exposes RESTful endpoints consumed by the React frontend, and:

- Accepts and returns JSON request/response bodies
- Returns appropriate HTTP status codes for success, validation, and authorization failures
- Delegates authentication to the JWT filter chain before any business logic executes

Security Configuration

Spring Security is configured with a custom JWT authentication filter and role-based authorization rules.

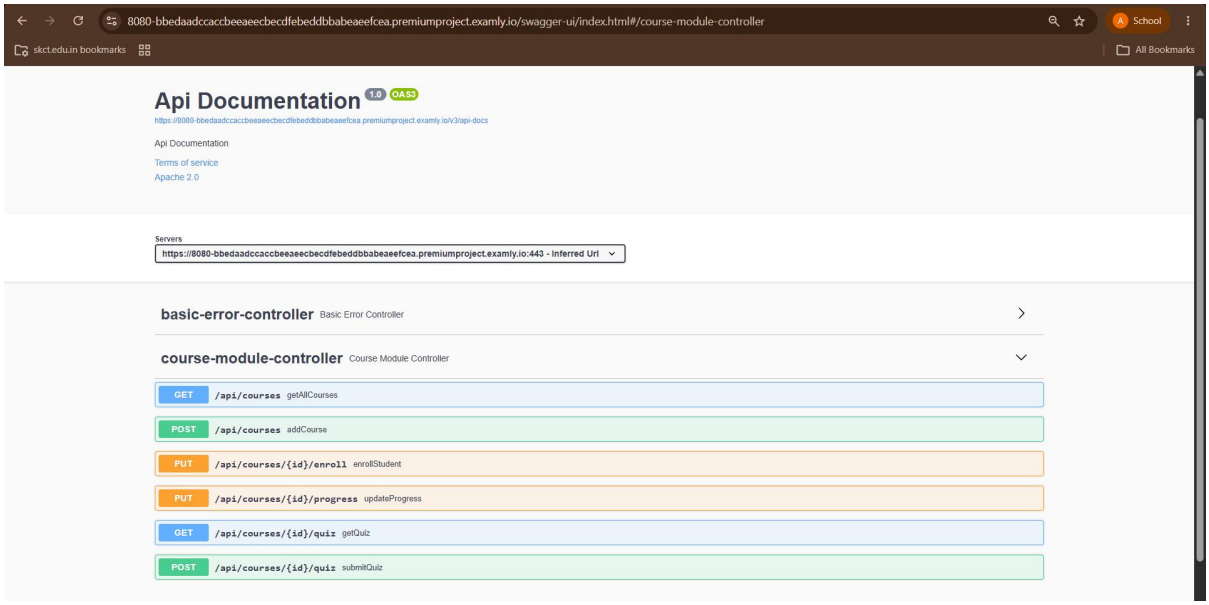
- JWT tokens signed using HS256, with role-specific expiration (8 hours for students, 12 hours for instructors, 6 hours for administrators)
- Refresh tokens valid for 7 days, with token blacklisting on logout or security events
- Password policy: minimum 8 characters, upper- and lower-case letters, a number and a special character, with reuse and periodic-expiry restrictions

CORS Configuration

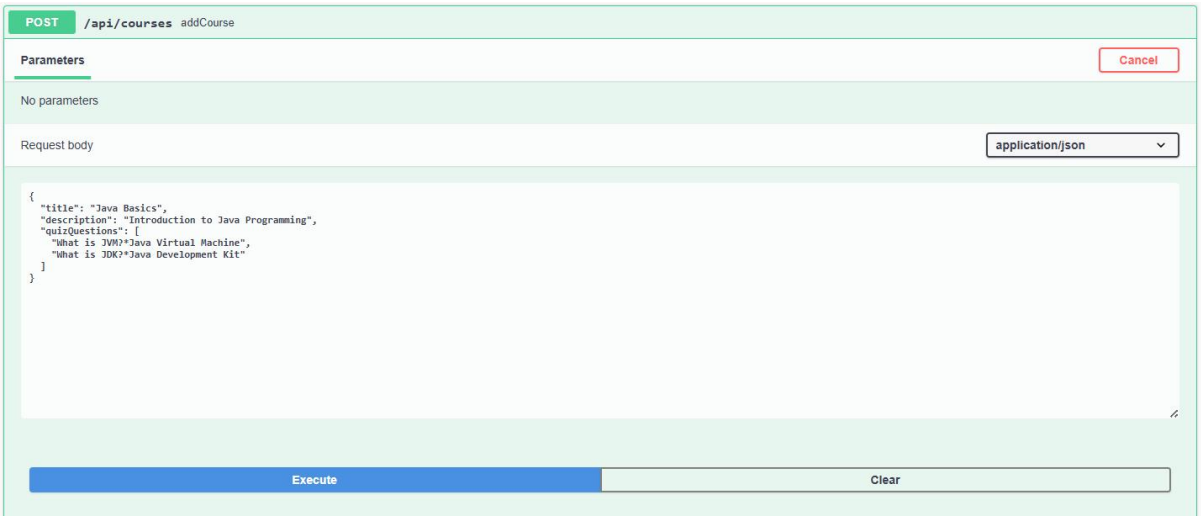
Cross-Origin Resource Sharing (CORS) is configured to allow secure communication between the frontend and backend running on different ports during development.

- React Frontend (Port 8081)
- Spring Boot Backend (Port 8080)
- Allowed HTTP methods: GET, POST, PUT, DELETE, OPTIONS

1. Swagger Home



2. POST (ADD COURSE)



Curl

```
curl -X POST "https://8080-bbedaadccaccbeeaeecbecdfebeddbbabeaeefcea.premiumproject.examly.io:443/api/courses" -H "accept: */*" -H "Content-Type: application/json" -d "{\"title\":\"Java Basics\",\"description\":\"Introduction to Java Programming\",\"quizQuestions\":[\"What is JVM?*Java Virtual Machine\",\"What is JDK?*Java Development Kit\"]}"
```

Request URL

```
https://8080-bbedaadccaccbeeaeecbecdfebeddbbabeaeefcea.premiumproject.examly.io:443/api/courses
```

Server response

Code	Details
200	Response body

```
{
  "id": 2,
  "title": "Java Basics",
  "description": "Introduction to Java Programming",
  "enrolledStudents": [],
  "progress": {},
  "quizQuestions": [
    "What is JVM?*Java Virtual Machine",
    "What is JDK?*Java Development Kit"
  ],
  "scores": {}
}
```

Download

Response headers

```
cache-control: no-cacheno-storemax-age=0must-revalidate
content-type: application/json
date: Fri31 Jul 2026 06:24:00 GMT
expires: 0
pragma: no-cache
strict-transport-security: max-age=15724800; includeSubDomains
vary: OriginAccess-Control-Request-MethodAccess-Control-Request-Headers
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 1; mode-block
```

Responses

Code	Description	Links
200	OK	*No links*

3. GET (Get All Courses)

course-module-controller Course Module Controller

GET /api/courses getAllCourses

Parameters | Cancel

No parameters

Execute | Clear

Responses

Curl

```
curl -X GET "https://8080-bbedaadccaccbeeaeecbecdfebeddbbabeaeefcea.premiumproject.examly.io:443/api/courses" -H "accept: */*"
```

Request URL

```
https://8080-bbedaadccaccbeeaeecbecdfebeddbbabeaeefcea.premiumproject.examly.io:443/api/courses
```

Server response

Code	Details
200	Response body

```
[
  {
    "id": 1,
    "title": "Java Basics",
    "description": "Introduction to Java Programming",
    "enrolledStudents": [],
    "progress": {},
    "quizQuestions": [
      "What is JVM?*Java Virtual Machine",
      "What is JDK?*Java Development Kit"
    ],
    "scores": {}
  },
  {
    "id": 2,
    "title": "Java Basics",
    "description": "Introduction to Java Programming",
    "enrolledStudents": [],
    "progress": {},
    "quizQuestions": [
      "What is JVM?*Java Virtual Machine",
      "What is JDK?*Java Development Kit"
    ],
    "scores": {}
  }
]
```

Download

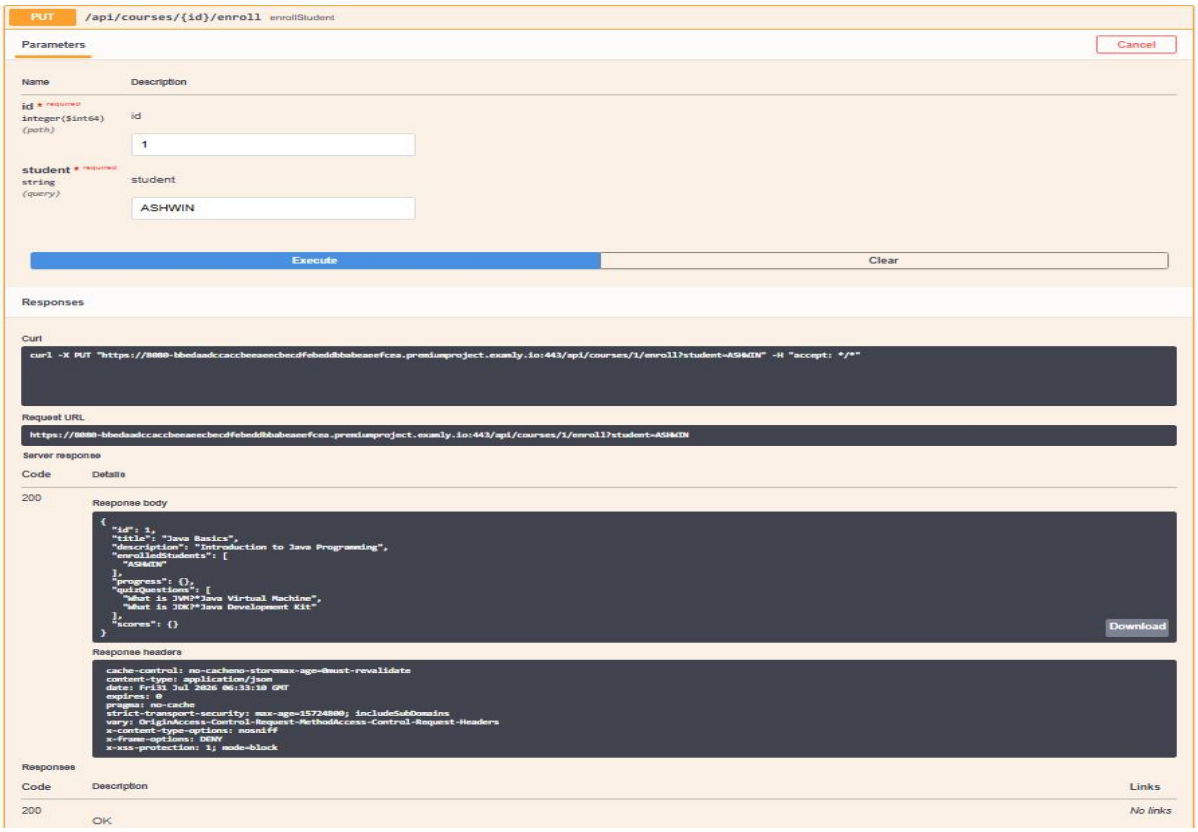
Response headers

```
cache-control: no-cacheno-storemax-age=0must-revalidate
content-type: application/json
date: Fri31 Jul 2026 06:29:37 GMT
expires: 0
pragma: no-cache
strict-transport-security: max-age=15724800; includeSubDomains
vary: OriginAccess-Control-Request-MethodAccess-Control-Request-Headers
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 1; mode-block
```

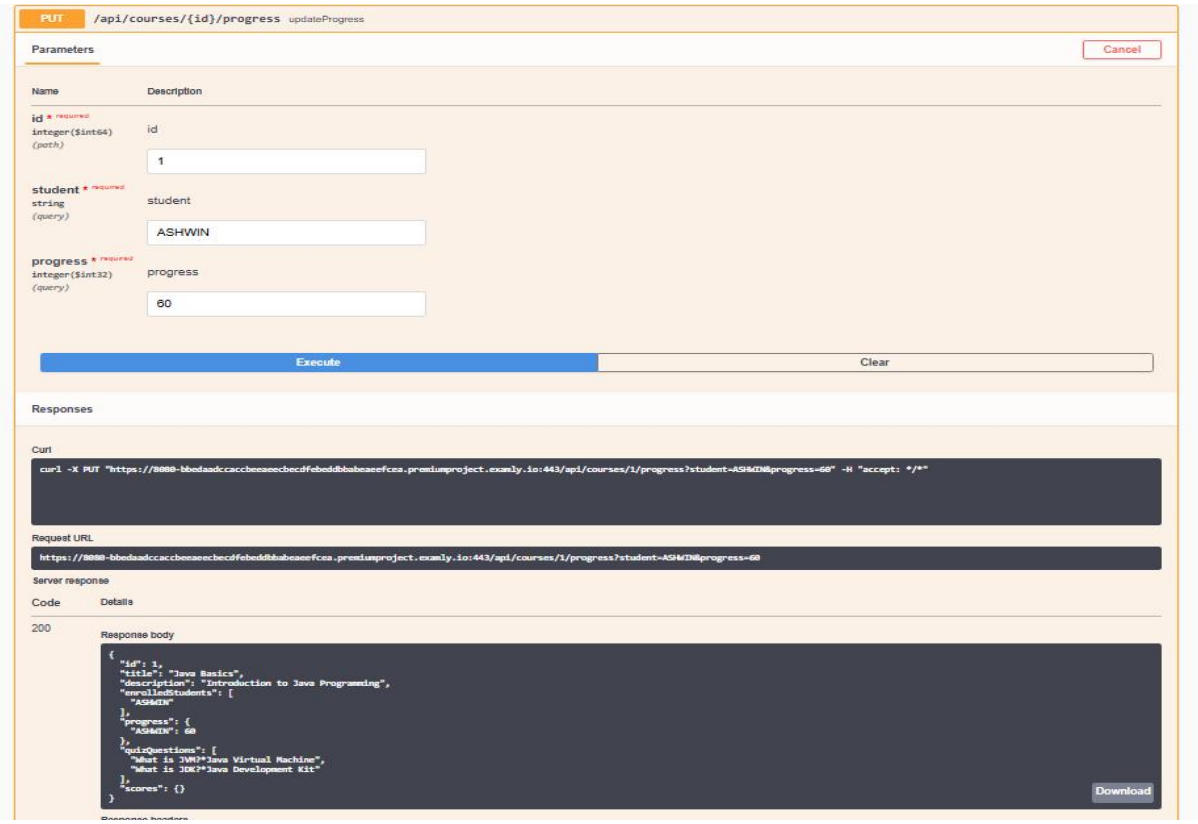
Responses

Code	Description	Links
200	OK	*No links*

4. PUT (Enroll Student)



5. PUT (Update Progress)



Code Details

200

Response body


```
{
  "id": 1,
  "title": "Java Basics",
  "description": "Introduction to Java Programming",
  "enrolledStudents": [
    "ASHWIN"
  ],
  "progress": {
    "ASHWIN": 60
  },
  "quizQuestions": [
    "What is JVM?*Java Virtual Machine",
    "What is JDK?*Java Development Kit"
  ],
  "scores": {}
}
```


Download

Response headers


```
cache-control: no-cacheno-storemax-age=0must-revalidate
content-type: application/json
date: Fri31 Jul 2026 06:35:40 GMT
expires: 0
pragma: no-cache
strict-transport-security: max-age=15724800; includeSubDomains
vary: OriginAccess-Control-Request-MethodAccess-Control-Request-Headers
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 1; mode=block
```


Responses

Code	Description	Links
200	OK Media type */* Controls Accept header. Example Value | Schema	*No links*
201	Created	*No links*


```
{
  "description": "string",
  "enrolledStudents": [
    "string"
  ],
  "id": 0,
  "progress": {
    "additionalProp1": 0,
    "additionalProp2": 0,
    "additionalProp3": 0
  },
  "quizQuestions": [
    "string"
  ],
  "scores": {
    "additionalProp1": 0,
    "additionalProp2": 0,
    "additionalProp3": 0
  },
  "title": "string"
}
```


6. GET (Get Quiz Questions)

GET /api/courses/{id}/quiz getQuiz

Parameters Cancel

Name	Description
id * required integer($int64) (path)	id 1

Execute Clear

Responses

Curl


```
curl -X GET "https://8080-bbedaadccaccbeeaeecbecdfebeddbbabeaeefcea.premiumproject.examly.io:443/api/courses/1/quiz" -H "accept: */*"
```


Request URL


```
https://8080-bbedaadccaccbeeaeecbecdfebeddbbabeaeefcea.premiumproject.examly.io:443/api/courses/1/quiz
```


Server response

Code Details

200

Response body


```
[
  "What is JVM?*Java Virtual Machine",
  "What is JDK?*Java Development Kit"
]
```


Download

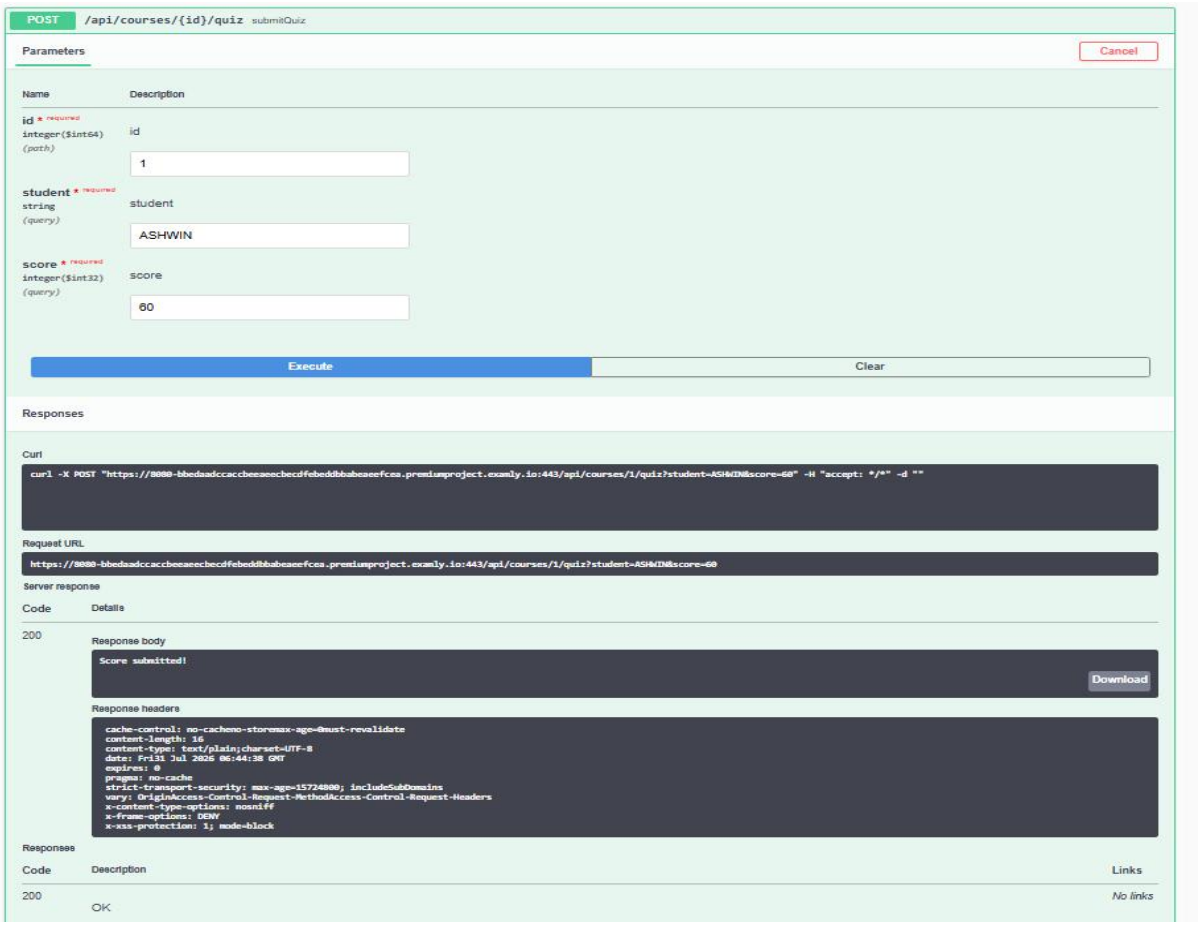
Response headers


```
cache-control: no-cacheno-storemax-age=0must-revalidate
content-type: application/json
date: Fri31 Jul 2026 06:41:47 GMT
expires: 0
pragma: no-cache
strict-transport-security: max-age=15724800; includeSubDomains
vary: OriginAccess-Control-Request-MethodAccess-Control-Request-Headers
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 1; mode=block
```


Responses

Code	Description	Links
200	OK	*No links*

7. POST (Submit Quiz Score)



CHAPTER - 10

API ENDPOINTS

Method	Endpoint	Description
POST	/auth/register	Register a new user
POST	/auth/login	Authenticate user and issue JWT
POST	/auth/logout	Invalidate current session token
GET	/users/profile	Retrieve authenticated user's profile
PUT	/users/profile	Update authenticated user's profile
POST	/courses	Create a new course
GET	/courses	Retrieve all available courses
GET	/courses/{id}	Retrieve specific course details
PUT	/courses/{id}	Update an existing course
DELETE	/courses/{id}	Archive a course
POST	/enrollments	Enroll a student in a course
GET	/enrollments/student/{studentId}	Get a student's enrollments
GET	/enrollments/course/{courseId}	Get a course's enrollments
PUT	/enrollments/{id}	Update enrollment status
POST	/assessments	Create a new assessment
GET	/assessments/course/{courseId}	Get assessments for a course
POST	/submissions	Submit an assessment
GET	/submissions/{id}	Get submission details

CHAPTER - 11

TESTING

Testing is an essential phase of the development lifecycle, ensuring the platform functions correctly and satisfies the specified requirements. The backend of the LMS Lite system was tested using JUnit and MockMvc, while the React frontend was tested using React Testing Library.

Testing performed using:

- JUnit — unit testing of Controller and Service methods
- MockMvc — simulating HTTP requests and verifying response status codes and JSON structure
- React Testing Library — component-level testing of the frontend dashboards
- Manual API testing through tools such as Postman during development

Areas Verified:

- Authentication and JWT token issuance / validation
- Role-based access control on protected endpoints
- Course, enrollment, and assessment CRUD operations
- Grade calculation and submission scoring logic
- Error handling for invalid input and unauthorized requests

The project targets a minimum of 85% code coverage across both the backend and frontend, along with periodic security and penetration testing, and load testing for the expected concurrent-user volumes described in the system's performance requirements.

CHAPTER - 12

SOFTWARE REQUIREMENTS

Software	**Version / Specification**
Operating System	Windows 10/11 or Linux
Java Development Kit (JDK)	JDK 17+
Spring Boot	3.x
Spring Security	Included with Spring Boot 3.x
React.js	Latest Stable Version
Maven	3.x
Database Server	Any JPA-compatible RDBMS
Spring Data JPA	Included with Spring Boot
IDE	Visual Studio Code / IntelliJ IDEA
Version Control	Git
API Testing Tool	Postman
Web Browser	Google Chrome / Microsoft Edge

CHAPTER - 13

SAMPLE JSON REQUEST & RESPONSE

Sample Request — Create Course (POST /courses)

{

"courseCode": "CS301",

"title": "Database Management Systems",

"description": "Introduction to relational database design and SQL",

"instructorId": 12,

"credits": 4,

"startDate": "2026-08-01",

"endDate": "2026-12-10",

"maxEnrollment": 60

}

Sample Response

{

"id": 101,

"courseCode": "CS301",

"title": "Database Management Systems",

"description": "Introduction to relational database design and SQL",

"instructorId": 12,

"credits": 4,

"startDate": "2026-08-01",

"endDate": "2026-12-10",

"status": "DRAFT",

"maxEnrollment": 60,

"createdDate": "2026-07-24T10:15:00"

}

CHAPTER - 14

CONCLUSION

The Learning Management System (LMS) Lite backend was successfully designed and implemented using the Spring Boot framework, following a layered architecture consisting of the Controller, Service, Repository, and Model components, secured throughout by Spring Security and JWT-based authentication. The application provides a reliable, role-aware solution for managing courses, enrollments, content, assessments, and grades through RESTful APIs.

The backend successfully performs all essential CRUD operations across the core academic entities, enforces a five-tier role-based access control model, and supports assessment creation with automated grading. Academic data is stored using Spring Data JPA against a JPA-compatible database, and design constraints such as FERPA-aligned data handling, audit logging, and input validation have been incorporated throughout the specification and implementation.

The REST APIs were structured for verification using JUnit and MockMvc on the backend and React Testing Library on the frontend, ensuring that endpoints behave correctly across the range of roles and scenarios the system supports. Overall, the project satisfies the functional and non-functional requirements defined in the system's Software Requirements Specification and establishes a strong, modular foundation for the complete LMS Lite platform, supporting future enhancements such as predictive analytics, AI-powered recommendations, and deeper institutional integrations.

CHAPTER - 15

FUTURE ENHANCEMENTS

The current version of LMS Lite focuses on the core backend functionality for course, assessment, and user management. Future versions can incorporate the following enhancements, in line with the platform's longer-term roadmap:

Phase 2

- Native mobile applications for iOS and Android
- AI-powered learning content recommendations
- Integration with virtual reality learning environments
- Blockchain-based credential verification
- Advanced proctoring and anti-cheating measures

Phase 3

- AI-driven personalized learning paths
- Advanced analytics with machine learning insights
- Integration with employer systems for workforce development
- Multi-language support for international programs
- Advanced accessibility features for diverse learners

Phase 4

- Virtual reality classroom experiences
- AI-powered teaching assistants
- Advanced competency-based learning management
- Integration with industry certification programs
- Global education network connectivity

Document Version Control

Version	Changes	Author
1.0	Initial project report creation for Learning Management System (LMS) Lite	Ashwin